Networking Fundamentals – 2

In the previous networking concepts, we looked at how devices communicate, how DNS resolves domain names, and how traffic moves through networks.

Now we will build on that foundation and understand some important production-level networking components:

- Forward Proxy
- Reverse Proxy
- API Gateway
- HTTP vs HTTPS
- SSL/TLS
- Symmetric and Asymmetric Cryptography
- Digital Certificates
- Certificate Authorities
- TLS Handshake
- ECDHE Key Exchange
- Certificate Management
- Let’s Encrypt, ACME and Certbot

---

1. Understanding a Proxy

A **proxy** is simply a system that stands between two communicating parties.

```
A → Proxy → B
```

The most important question when understanding a proxy is:

> Who is the proxy representing?

Depending on the answer, we broadly get two types:

1. Forward Proxy
2. Reverse Proxy

---

2. Forward Proxy

A **forward proxy represents the client**.

Example:

```
Employee Laptop
      ↓
Corporate Proxy
      ↓
Internet
```

Instead of the employee directly connecting to a website, the request first goes through the organization's proxy.

```
Client → Forward Proxy → Server
```

The destination website may see the connection coming from the proxy rather than directly from the employee's machine.

The client usually knows that it is using the proxy.

Common Uses

Forward proxies can be used for:

- Corporate internet filtering
- Access control
- Privacy
- Monitoring
- Caching

- Restricting access to particular websites

---

3. Reverse Proxy

A **reverse proxy represents the server side**.

```
Client
  ↓
Reverse Proxy
  ↓
Backend Server
```

Suppose a user visits:

```
coderarmy.in
```

The actual architecture may be:

```
coderarmy.in
     ↓
   Nginx
     ↓
Spring Boot
```

The user only knows about `coderarmy.in`.

They do not need to know:

- Where the Spring Boot application is running
- What port it is using
- How many backend servers exist
- What the internal IP addresses are

The backend infrastructure can remain hidden behind the reverse proxy.

---

4. Why Use a Reverse Proxy?

Imagine your Spring Boot application is running internally on:

```
10.0.1.20:8080
```

Instead of exposing this application directly to the Internet, we can expose Nginx.

```
Internet
   |
   | HTTPS :443
   ↓
 Nginx
   |
   | HTTP :8080
   ↓
Spring Boot
```

Nginx receives the public request and forwards it to the internal application.

This gives us a clean separation between:

```
Internet-facing infrastructure
          ↓
Application infrastructure
```

A reverse proxy can also be used for things such as:

- TLS termination
- Routing
- Load balancing
- Hiding backend servers
- Centralized request handling

---

5. Basic Nginx Reverse Proxy Configuration

A very simple conceptual Nginx configuration may look like:

```
server {
    listen 80;

    location / {
        proxy_pass http://localhost:8080;
    }
}
```

Here:

```
Client
   ↓
Nginx :80
   ↓
Spring Boot :8080
```

Nginx receives the request on port `80` and forwards it to the application running on port `8080`.

---

6. API Gateway Fundamentals

A reverse proxy becomes even more useful when an application grows into multiple services.

With a simple monolithic application:

```
Client
   ↓
Spring Boot Application
```

But with microservices, we may have:

```
User Service
Product Service
Order Service
Payment Service
```

```
Notification Service
Review Service
```

Without another layer, the frontend might need to know separate addresses such as:

```
users.company.com
products.company.com
orders.company.com
payments.company.com
```

Technically this can work, but it introduces unnecessary complexity.

Instead, applications commonly expose a single entry point through an **API Gateway**.

---

7. What Does an API Gateway Do?

The client interacts with one endpoint:

```
api.coderarmy.in
```

Different API paths can then be forwarded to different services.

```
/api/users
     ↓
User Service
```

```
/api/products
     ↓
Product Service
```

```
/api/orders
     ↓
Order Service
```

Architecture:

```
                Client
                   |
                   ↓
             API Gateway
            /      |      \
           /       |       \
          ↓        ↓        ↓
    Users    Products   Orders
    Service  Service    Service
```

The frontend gets one consistent API endpoint while the gateway understands where each request should go internally.

---

8. API Gateway Routing

Suppose the client sends:

```
GET /api/products/10
```

The API Gateway examines the route:

```
/products
```

and forwards the request to:

```
Product Service
```

Similarly:

```
POST /api/orders
```

can be forwarded to:

```
Order Service
```

The client doesn’t need to know the location of individual services.

---

9. Authentication at the API Gateway

Suppose every microservice independently performs:

```
JWT Verification
Authentication
Authorization
```

This may result in repeated infrastructure logic across many services.

Depending on the architecture, some of this work can instead happen at the API Gateway.

Example:

```
Client
   |
   | JWT
   ↓
API Gateway
   |
   | Verify Token
   |
   ├── Invalid → 401
   |
   └── Valid
         ↓
    Order Service
```

The exact security architecture varies between systems.

The important idea is that the API Gateway can act as a centralized point for common infrastructure concerns.

---

10. Rate Limiting

Imagine somebody sends:

```
100,000 requests/second
```

to:

```
/api/login
```

Allowing unlimited requests could overload the application or make attacks easier.

The gateway can enforce limits such as:

```
100 requests/minute/user
```

or:

```
10 login attempts/minute/IP
```

This is called **rate limiting**.

It helps protect backend services from excessive traffic.

---

11. API Gateway + Load Balancer Architecture

A real application might have multiple instances of every service.

```
                    API Gateway
                         |
              +----------+----------+
              |          |          |
              ↓          ↓          ↓
        User Service  Product Service  Order Service
              |          |          |
```

```
          ↓              ↓              ↓
    Load Balancer  Load Balancer   Load Balancer
       /   \          /   \          /   \
      ↓     ↓        ↓     ↓        ↓     ↓
     U1     U2       P1    P2       O1    O2
```

The responsibilities are different.

API Gateway

Understands application-level API routes.

Example:

```
/api/products → Product Service
```

Load Balancer

Distributes requests among multiple instances of the same service.

Example:

```
Product Service
      ↓
Load Balancer
   /       \
  ↓         ↓
 P1         P2
```

---

12. HTTP vs HTTPS

Our application request flow may now look like:

```
Browser
   ↓
  DNS
   ↓
Reverse Proxy / API Gateway / Load Balancer
```

```
   ↓
Application
```

But there is an important problem.

Suppose a browser sends:

```
POST /login
```

with:

```
username=rohit
password=mysecret123
```

We do not want sensitive information travelling across the Internet without transport-level protection.

This is where HTTPS becomes important.

---

13. What is HTTP?

HTTP stands for:

> Hypertext Transfer Protocol

HTTP defines how web clients and servers communicate.

Example request:

```
GET /products/10 HTTP/1.1
Host: api.coderarmy.in
```

Example response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
```

```
  "id": 10,
  "name": "Laptop"
}
```

HTTP defines concepts such as:

- GET
- POST
- PUT
- DELETE
- Headers
- Request body
- Response body
- Status codes

HTTP gives us the communication rules.

---

14. What HTTP Does Not Provide by Itself

Plain HTTP does not provide TLS encryption for application data while it is travelling across the network.

Conceptually:

```
Browser
   |
   | username=rohit
   | password=hello123
   ↓
Server
```

Network traffic may travel through many devices before reaching the destination.

If somebody is able to intercept unprotected traffic, its contents may potentially be read or modified.

That becomes extremely dangerous for data such as:

- Passwords
- Authentication tokens
- Cookies
- Credit card information
- Personal information
- API requests

---

15. HTTPS

HTTPS is essentially:

```
HTTP
 +
TLS
 =
HTTPS
```

HTTPS is not a completely different replacement for HTTP.

Instead, HTTP communication is protected by TLS.

Conceptually:

```
HTTP Request
      ↓
TLS Encryption
      ↓
Encrypted Network Data
      ↓
Server
      ↓
TLS Decryption
      ↓
HTTP Request
```

---

16. Default HTTP and HTTPS Ports

Common default ports are:

Protocol	Default Port
HTTP	80
HTTPS	443

---

17. What Does HTTPS Give Us?

HTTPS through TLS provides three major security properties.

17.1 Confidentiality

Other people should not be able to understand the data being transmitted.

For example:

```
password=hello123
```

TLS encrypts the communication before sending it across the network.

An observer should not simply be able to read the original password.

17.2 Integrity

Data should not be silently modified while travelling through the network.

For example, an attacker should not be able to change:

```
Pay ₹100
```

into:

```
Pay ₹100000
```

without the modification being detected.

TLS provides integrity protection for transmitted data.

17.3 Authentication

The browser also needs confidence that:

```
https://google.com
```

is actually communicating with a server authorized for `google.com`.

This is where **digital certificates** and **Certificate Authorities** become important.

18. Why Does HTTPS Need a Certificate?

When you connect to:

```
https://coderarmy.in
```

the server presents a digital certificate.

At a high level, the certificate helps establish that:

> A particular public key is authorized for a particular hostname through a trusted certificate chain.

The browser verifies things such as:

- Is the certificate currently valid?
- Has it expired?
- Does the hostname match?
- Is the certificate chain trusted?
- Are the cryptographic signatures valid?
- Are relevant certificate security requirements satisfied?

Only after these checks can the browser trust the server identity.

19. HTTP to HTTPS Redirect

Production websites commonly redirect HTTP traffic to HTTPS.

For example:

```
http://coderarmy.in
```

redirects to:

```
https://coderarmy.in
```

Architecture:

```
User
 |
 | HTTP :80
 ↓
Nginx
 |
 | 301 / 308 Redirect
 ↓
https://coderarmy.in
 |
 | HTTPS :443
 ↓
Application
```

This ensures normal application traffic uses HTTPS.

---

20. TLS Termination

Remember the reverse proxy architecture:

```
Browser
   |
   | HTTPS :443
   ↓
 Nginx
```

```
   |
   | HTTP :8080
   ↓
Spring Boot
```

Nginx may contain:

```
Certificate
Private Key
TLS Configuration
```

Nginx receives the HTTPS connection and decrypts the traffic before forwarding it to the backend.

This is known as:

> **TLS Termination**

The backend connection may use plain HTTP inside a trusted private network, depending on the architecture.

In other environments, TLS may also be used between internal services.

---

21. SSL vs TLS

You will frequently hear terms such as:

```
SSL Certificate
SSL Encryption
```

However, modern HTTPS uses **TLS**.

SSL refers to older protocols that have been replaced by TLS.

The industry still commonly uses the phrase **SSL certificate**, but technically modern secure web communication uses TLS.

---

22. Why Do We Need Encryption?

Suppose the browser sends:

```
POST /login
```

with:

```
username=rohit
password=secret123
```

Without TLS:

```
Browser
   |
   | username=rohit
   | password=secret123
   ↓
Internet
   ↓
Server
```

What we want instead is:

```
Browser
   |
   | encrypted data
   | 8as7df8as7d...
   ↓
Internet
   ↓
Server
```

Even if somebody observes the traffic, they should not be able to understand the original data.

To understand how TLS achieves this, we first need to understand cryptography.

---

23. Symmetric Encryption

Symmetric encryption uses the **same secret key** for encryption and decryption.

Suppose both sides know:

```
SECRET_KEY = ABC123
```

Encryption:

```
Hello Rohit
     ↓
Encrypt using ABC123
     ↓
X8#29KD!...
```

Decryption:

```
X8#29KD!...
     ↓
Decrypt using ABC123
     ↓
Hello Rohit
```

Conceptually:

```
                Key K

Plaintext
    ↓
Encrypt using K
    ↓
Ciphertext
    ↓
Decrypt using K
    ↓
Plaintext
```

---

24. Why Symmetric Encryption is Useful

Symmetric cryptography is very efficient.

That makes it suitable for protecting large amounts of data such as:

- HTML
- JSON
- Images
- API requests
- Passwords
- Videos
- File downloads

Therefore, TLS ultimately uses **symmetric traffic keys** for protecting the actual application data.

---

25. The Symmetric Key Exchange Problem

Symmetric encryption creates one major problem.

Both sides need the same secret.

```
Browser                      Server

Secret = XYZ                Secret = XYZ
```

But how do they both obtain `XYZ`?

The browser cannot simply send:

```
"Our secret key is XYZ"
```

over the Internet.

An attacker observing the connection could learn the secret.

This creates the fundamental **key exchange problem**:

> How can two parties that have never communicated before establish shared cryptographic secrets over an insecure network?

This is where public-key cryptography becomes useful.

---

26. Asymmetric / Public-Key Cryptography

Instead of one secret key, asymmetric cryptography uses a pair:

```
Public Key
Private Key
```

They are mathematically related.

The important rule is:

```
Public Key  → Can be shared
Private Key → Must remain secret
```

Example:

```
Server

Public Key  → Everyone can know
Private Key → Only server should know
```

---

27. Simplified Public-Key Encryption Model

A simplified way to understand public-key encryption is:

```
Encrypt using Public Key
          ↓
Decrypt using Private Key
```

If the server publishes:

```
SERVER_PUBLIC_KEY
```

the browser could theoretically encrypt information for the server.

```
Secret
   ↓
Server Public Key
   ↓
Encrypted Secret
```

Only someone possessing the corresponding private key could decrypt it.

```
Browser
   |
   | Encrypted information
   ↓
Internet
   |
   | Attacker sees ciphertext
   ↓
Server
   |
   | Private Key
   ↓
Original information recovered
```

---

28. Why Not Encrypt Everything Using Public-Key Cryptography?

Public-key cryptography is computationally more expensive than symmetric cryptography.

Imagine sending a:

```
2 GB video
```

Using expensive public-key operations for every piece of data would be inefficient.

TLS therefore combines both approaches.

```
Public-Key Cryptography
          ↓
Authentication + Key Agreement

          THEN

Symmetric Cryptography
          ↓
Fast Encrypted Communication
```

Core Idea

```
Public-key cryptography
→ Helps establish trust and shared secrets

Symmetric cryptography
→ Protects the actual application traffic efficiently
```

This combination is known as a **hybrid cryptographic system**.

---

29. Encryption Alone Does Not Prove Identity

Suppose you visit:

```
https://coderarmy.in
```

and someone gives your browser a public key.

How does the browser know that the key actually belongs to Coder Army?

An attacker could potentially sit between the browser and server.

```
Browser
   ↓
Attacker
   ↓
CoderArmy Server
```

The browser asks for the server's cryptographic information.

The attacker responds using:

```
ATTACKER_PUBLIC_KEY
```

while pretending that it belongs to Coder Army.

The browser could then establish an encrypted connection with the attacker.

The connection would be encrypted, but with the wrong party.

This is a **Man-in-the-Middle attack**, commonly abbreviated as:

```
MITM
```

Therefore:

> Encryption alone is not enough. We also need identity verification.

---

30. Digital Certificates

Instead of the server simply saying:

> Trust me, this public key belongs to me.

the server provides a **digital certificate**.

A simplified certificate may contain:

```
Domain:
coderarmy.in
```

```
Public Key:
XYZ123...

Valid From:
...

Valid Until:
...

Issuer:
Certificate Authority

Digital Signature:
...
```

A certificate binds together information such as:

```
Hostname
   +
Public Key
   +
Identity / Validation Information
```

---

31. Important: Certificate vs Private Key

A digital certificate does **not** contain the server's private key.

The private key must remain secret on the server.

```
Certificate
    ↓
Can be public

Private Key
    ↓
Must remain secret
```

Anyone can download a website's public certificate.

That does not allow them to impersonate the server because they still don't possess the corresponding private key.

---

32. What is Inside a Certificate?

A simplified certificate contains information such as:

```
Certificate
├── Subject / hostname information
├── Public key
├── Issuer
├── Validity period
├── Serial number
├── Extensions
└── Digital signature
```

When the browser accesses:

```
https://example.com
```

the server sends its certificate.

The browser can then determine whether the certificate authorizes the hostname it is trying to reach.

---

33. Certificate Authority — CA

Who decides whether a certificate should be trusted?

Certificates are issued and cryptographically signed through organizations known as **Certificate Authorities**, or CAs.

Examples include:

- Let’s Encrypt
- DigiCert
- Sectigo
- GlobalSign

A Certificate Authority follows its validation policies before issuing certificates.

For a domain-validated certificate, this generally involves confirming control of the requested domain.

Conceptually:

```
coderarmy.in
     |
     | Request Certificate
     ↓
Certificate Authority
     |
     | Validate Domain Control
     ↓
Sign Certificate
```

The server can then present:

```
Certificate
     ↓
Signed through trusted CA infrastructure
```

---

34. Why Does the Browser Trust a CA?

Operating systems and browsers maintain a collection of trusted root certificates.

Conceptually:

```
Browser / OS Trust Store

✓ Root CA A
✓ Root CA B
✓ Root CA C
✓ ...
```

When the browser receives a website certificate, it attempts to verify a cryptographic chain leading back to one of these trusted roots.

---

35. Certificate Chain

A website certificate is usually not signed directly by a root certificate.

Instead, the structure is often:

```
Root CA
   |
   | Signs
   ↓
Intermediate CA
   |
   | Signs
   ↓
Website Certificate
```

Example:

```
Trusted Root
     ↓
Intermediate CA
     ↓
coderarmy.in Certificate
```

Root private keys are extremely valuable.

Keeping them more isolated reduces risk.

Intermediate CAs can therefore perform most day-to-day certificate issuance while root keys remain more heavily protected.

---

36. Browser Certificate Verification

Suppose the browser receives:

```
coderarmy.in Certificate
```

It may verify the chain conceptually as:

```
Website Certificate
       |
       | Signed by?
```

```
       ↓
Intermediate CA
       |
       | Signed by?
       ↓
Trusted Root CA
```

If the root exists in the browser or operating system trust store and the remaining validation checks pass:

```
✓ Trusted Certificate Chain
```

---

37. What Does the Browser Verify?

When visiting:

```
https://coderarmy.in
```

the browser broadly validates several things.

Domain Match

The certificate must authorize:

```
coderarmy.in
```

and not some unrelated domain such as:

```
randomwebsite.com
```

Validity Period

Certificates have a validity period.

```
Valid From: ...
Valid Until: ...
```

The certificate should not be:

- Expired
- Not yet valid

Signature Chain

The cryptographic signatures in the certificate chain must verify correctly.

Trusted Issuer

The chain should terminate in a root certificate trusted by the browser or operating system.

Other Security Checks

Modern certificate validation can also consider:

- Certificate extensions
- Key usage
- Security algorithms
- Certificate policies
- Revocation mechanisms where applicable

---

38. Public Key vs Certificate

These two concepts are different.

Public Key

A cryptographic key that can safely be shared.

Certificate

A digitally signed structure that effectively says:

> This public key is authorized for this identity/hostname, and here is cryptographic evidence supporting that relationship.

A public key alone does not establish identity.

The certificate provides the identity and trust context around that public key.

---

39. TLS Handshake

We now have the major pieces required to understand TLS:

- Symmetric cryptography
- Public/private keys
- Digital certificates
- Certificate Authorities
- Certificate chains

Suppose:

```
Browser                    Server
   ?                          ?
```

They have never communicated before.

They want to reach:

```
Browser                    Server

         Secure Connection
<============================>
```

During the TLS handshake, two important things need to happen:

1. The browser verifies the server's identity.

2. Both sides establish shared cryptographic traffic secrets.

After that, normal HTTP communication can begin securely.

---

40. TLS Handshake — Step 1: ClientHello

The browser connects to:

```
https://coderarmy.in
```

For HTTP/1.1 or HTTP/2, a TCP connection is normally established first.

The browser then sends a TLS message called:

```
ClientHello
```

Conceptually:

```
Browser → Server

Hello.

Here are:
- TLS versions I support
- Cryptographic algorithms I support
- The hostname I want
- Key exchange information
- Other TLS information
```

A simplified ClientHello can contain:

```
Supported TLS Versions
Supported Cipher Suites
Server Name
Key Share
...
```

The hostname is important because the same server or IP address may host multiple HTTPS websites.

---

41. Why Does the Browser Send Supported Algorithms?

The browser and server need to agree on cryptographic algorithms that both understand.

For example, the browser may support:

```
AES-128-GCM
AES-256-GCM
ChaCha20-Poly1305
```

while the server may support:

```
AES-128-GCM
ChaCha20-Poly1305
```

They need to select compatible cryptographic parameters.

---

42. TLS Handshake — Step 2: ServerHello

The server responds with:

```
ServerHello
```

Conceptually:

```
Server → Browser

We will use:
- TLS 1.3
- Selected cryptographic algorithms
- Selected key-exchange parameters
```

```
Here is my key share.
```

The browser and server now have the necessary public key-exchange information to begin deriving shared secrets.

---

43. TLS Handshake — Step 3: Server Certificate

The server provides its certificate chain.

Example:

```
Server → Browser

Certificate:
coderarmy.in

Public Key: ...
Issuer: ...
Validity: ...
Signature: ...
```

This gives the browser the information required to verify the server's identity.

---

44. TLS Handshake — Step 4: Browser Verifies the Certificate

Before trusting the server, the browser verifies things such as:

```
Does the certificate authorize coderarmy.in?

Is the certificate valid?

Is the signature chain correct?

Does the chain terminate at a trusted root?
```

If certificate validation fails, the browser may display errors such as:

```
Your connection is not private

NET::ERR_CERT...
```

This means TLS certificate validation could not be completed successfully.

---

45. TLS Handshake — Step 5: Server Proves Private Key Ownership

There is another important issue.

A certificate is public.

Anyone can download and copy:

```
coderarmy.in Certificate
```

Therefore, simply presenting the certificate is not enough.

The server must prove:

> I possess the private key corresponding to the public key contained in this certificate.

Modern TLS does this using a cryptographic signature over handshake data.

Conceptually:

```
Server
   |
Private Key
   |
Signs Handshake Information
   ↓
Browser
   |
```

```
Verifies Signature
using Certificate Public Key
```

If verification succeeds:

```
✓ Server possesses corresponding private key
```

This is an extremely important part of server authentication.

---

46. TLS Handshake — Step 6: Establishing a Shared Secret

Older explanations of TLS often describe the browser doing something like:

```
Create session key
      ↓
Encrypt it using server public key
      ↓
Send encrypted key
      ↓
Server decrypts using private key
```

This describes older RSA-based key exchange approaches.

Modern **TLS 1.3** normally uses ephemeral Diffie-Hellman key agreement, commonly:

> **ECDHE — Elliptic Curve Diffie-Hellman Ephemeral**

ECDHE allows the browser and server to independently derive the same shared secret without sending that secret directly across the network.

---

47. Understanding Diffie-Hellman Using Colours

Before looking at the mathematics, imagine both sides publicly agree on:

```
Starting Colour = Yellow
```

Everybody knows yellow, including an attacker.

Browser

The browser secretly chooses:

```
Private Colour = Blue
```

It combines:

```
Yellow + Blue
```

and sends that public mixture to the server.

Server

The server secretly chooses:

```
Private Colour = Red
```

It combines:

```
Yellow + Red
```

and sends that public mixture to the browser.

Now:

```
Browser                          Server

Private: Blue                    Private: Red
```

```
Yellow + Blue  ------------------->

               <------------------- Yellow + Red
```

Browser Computes the Final Mixture

The browser receives:

```
Yellow + Red
```

and adds its private blue:

```
Yellow + Red + Blue
```

Server Computes the Final Mixture

The server receives:

```
Yellow + Blue
```

and adds its private red:

```
Yellow + Blue + Red
```

Both sides now have equivalent mixtures:

```
Browser = Yellow + Red + Blue

Server  = Yellow + Blue + Red
```

Therefore:

```
Browser Result = Server Result
```

The attacker only saw:

```
Yellow

Yellow + Blue

Yellow + Red
```

but cannot easily recover the private blue or red.

> The colour example is only an analogy. Real ECDHE uses mathematics, not colours.

The central idea is:

```
Common Public Value
        ↓
Each side applies private value
        ↓
Exchange public results
        ↓
Each side combines received value
with its own private value
        ↓
Both derive same secret
```

---

48. Translating the Idea into ECDHE

ECDHE uses mathematical points on an elliptic curve.

You do not need to understand elliptic curve geometry to understand the networking concept.

We only need one conceptual operation:

```
Private Number × Public Starting Point
```

This operation has an important property.

```
Forward calculation
→ Easy

Reverse calculation
→ Extremely difficult
```

---

49. Public Starting Point

Both sides know a public starting point:

```
G = Public Starting Point
```

This is similar to the public yellow colour from the previous analogy.

`G` is not secret.

---

50. Browser Creates a Temporary Key Pair

The browser randomly chooses a temporary private value:

```
a = Browser Private Value
```

It calculates:

```
A = a × G
```

Where:

```
a = Browser Private Value
G = Public Starting Point
A = Browser Public Value
```

The browser keeps:

```
a
```

secret.

It sends:

```
A
```

to the server.

```
Browser Private Value: a → Never Sent
Browser Public Value:  A → Sent
```

---

51. Server Creates a Temporary Key Pair

The server randomly chooses:

```
b = Server Private Value
```

and calculates:

```
B = b × G
```

The server keeps:

```
b
```

secret and sends:

```
B
```

to the browser.

```
Server Private Value: b → Never Sent
Server Public Value:  B → Sent
```

Now:

```
Browser                                  Server

Private: a                              Private: b

Public:
A = a × G                               B = b × G


A = a × G  --------------------------->

           <--------------------------- B = b × G
```

---

52. Browser Calculates the Shared Secret

The browser receives:

```
B = b × G
```

and combines it with its private value `a`.

```
Shared Secret = a × B
```

Because:

```
B = b × G
```

we get:

```
a × B
= a × (b × G)
= a × b × G
```

So the browser calculates conceptually:

```
Shared Secret = abG
```

---

53. Server Calculates the Same Shared Secret

The server receives:

```
A = a × G
```

and combines it with its private value `b`.

```
Shared Secret = b × A
```

Since:

```
A = a × G
```

we get:

```
b × A
= b × (a × G)
= b × a × G
```

So the server obtains:

```
Shared Secret = baG
```

Conceptually:

```
abG = baG
```

Therefore:

```
Browser → Same Shared Secret
Server  → Same Shared Secret
```

Neither side needed to send the shared secret directly through the network.

---

54. Why Can’t an Attacker Calculate the Shared Secret?

An attacker can observe public information such as:

```
G

A = a × G

B = b × G
```

But the attacker does not know:

```
a
```

```
b
```

To calculate the shared secret, they would need something equivalent to:

```
a × B
```

or:

```
b × A
```

The attacker might try to recover `a` from:

```
A = a × G
```

However, elliptic-curve cryptography is designed around the fact that:

```
Given a and G:
Calculating A is easy.

Given A and G:
Recovering a is computationally infeasible.
```

This is related to the:

> **Elliptic Curve Discrete Logarithm Problem**

Conceptually:

```
Easy:

Private Value + Public Point
           ↓
      Public Value
```

But:

```
Extremely Difficult:

Public Value + Public Point
          ↓
Find Private Value
```

This one-way mathematical property is what protects the private values.

---

55. Why Is It Called ECDHE?

ECDHE stands for:

```
Elliptic Curve
Diffie-Hellman
Ephemeral
```

The **ephemeral** part means temporary key-exchange values are created for the connection.

Conceptually:

```
Browser Temporary Private Value
Server Temporary Private Value
```

These are separate from the long-term private key associated with the server certificate.

The certificate key helps authenticate the server.

The temporary ECDHE values help establish shared secrets for the connection.

---

56. TLS Handshake — Step 7: Traffic Keys Are Derived

The raw ECDHE shared secret is not simply used directly as the encryption key for everything.

TLS performs a key derivation process using the shared secret and handshake information.

Conceptually:

```
Shared Secret
      ↓
Key Derivation
      ↓
Client Traffic Key
Server Traffic Key
Other TLS Secrets
```

Now both the browser and server possess the cryptographic keys required to protect the connection.

Separate traffic keys can be used for different communication directions.

```
Client → Server
uses appropriate client traffic secret/key

Server → Client
uses appropriate server traffic secret/key
```

Therefore, both the request and the response are encrypted.

---

57. TLS Handshake — Step 8: Handshake Finishes

Both sides verify that they successfully completed the same handshake and derived the expected cryptographic secrets.

After this:

```
TLS Secure Channel Established
```

The browser can now begin sending application data.

---

58. TLS Handshake — Step 9: HTTP Runs Inside TLS

The actual HTTP request can now be transmitted.

Instead of application data such as:

```
GET /products
Authorization: Bearer xyz
```

travelling openly over the network, TLS protects the records carrying the request.

Conceptually:

```
HTTP Request

GET /products
      ↓
TLS Encryption
      ↓
8F3A9C...encrypted...
      ↓
Internet
      ↓
Server
      ↓
TLS Decryption
      ↓
GET /products
```

The server processes the request.

The response travels back through the same protected TLS connection.

```
Server Response
      ↓
TLS Encryption
      ↓
Internet
      ↓
Browser
      ↓
TLS Decryption
```

```
      ↓
HTTP Response
```

HTTPS therefore protects communication in **both directions**.

59. Complete TLS Handshake Flow

A simplified overall flow is:

```
Browser                                      Server
   |                                           |
   |------------ ClientHello ----------------->|
   |                                           |
   |<----------- ServerHello -----------------|
   |                                           |
   |<----------- Certificate -----------------|
   |                                           |
   |       Verify Certificate                  |
   |       Verify Server Identity              |
   |                                           |
   |<----- Server proves private key ----------|
   |                                           |
   |<------ ECDHE public key exchange -------->|
   |                                           |
   |    Both derive shared secret              |
   |                                           |
   |    Derive symmetric traffic keys          |
   |                                           |
   |====== TLS Secure Channel Established ======|
   |                                           |
   |---------- Encrypted HTTP Request --------->|
   |                                           |
   |<--------- Encrypted HTTP Response --------|
```

60. Certificate Management

Understanding TLS is only one part of the job.

A DevOps engineer also needs to manage certificates in production.

Suppose the architecture is:

```
https://coderarmy.in
        ↓
      Nginx
        ↓
Spring Boot :8080
```

Nginx needs two important things to terminate TLS:

```
Certificate
Private Key
```

---

61. Certificate vs Private Key on the Server

Certificate

```
Certificate
    ↓
Contains public-key and identity information
    ↓
Can be publicly shared
```

Private Key

```
Private Key
    ↓
Must remain secret
    ↓
Used to prove ownership of the certificate key pair
```

If the private key is compromised, an attacker may potentially impersonate the server depending on the circumstances.

Therefore, private keys must be protected carefully.

---

62. Common Certificate Files

On Linux servers you may encounter files such as:

```
certificate.crt
certificate.pem
fullchain.pem
privkey.pem
```

The exact filenames depend on:

- Certificate provider
- Web server
- Certificate management software
- Deployment setup

With Let’s Encrypt, certificates are commonly stored under a path such as:

```
/etc/letsencrypt/live/coderarmy.in/
```

Files may include:

```
cert.pem
chain.pem
fullchain.pem
privkey.pem
```

For a typical web server configuration, two particularly important files are often:

```
fullchain.pem
privkey.pem
```

---

63. Nginx HTTPS Configuration

A conceptual Nginx HTTPS configuration might look like:

```
server {
    listen 443 ssl;

    server_name coderarmy.in;

    ssl_certificate /path/fullchain.pem;
    ssl_certificate_key /path/privkey.pem;

    location / {
        proxy_pass http://localhost:8080;
    }
}
```

Flow:

```
Browser
   |
   | HTTPS :443
   ↓
Nginx
   |
   | HTTP :8080
   ↓
Spring Boot
```

Nginx handles TLS and forwards the application request to Spring Boot.

---

64. Certificates Expire

TLS certificates are not generally issued forever.

They have limited validity periods.

Example:

```
Valid From:
2026-08-01

Valid Until:
2026-10-30
```

After expiry:

```
Browser
   ↓
Expired Certificate
   ↓
Security Warning
```

Users may see messages such as:

```
Your connection is not private

Certificate expired
```

Therefore:

> Certificate renewal is an important operational responsibility.

---

65. Let’s Encrypt

To serve:

```
https://coderarmy.in
```

you need a certificate trusted by browsers.

Certificates can be obtained through different public Certificate Authorities.

One widely used option is:

> **Let’s Encrypt**

Let’s Encrypt is a public Certificate Authority that provides domain-validated TLS certificates at no charge.

Conceptually:

```
Your Server
     ↓
Let's Encrypt
```

```
     ↓
Prove Domain Control
     ↓
Certificate Issued
```

It is particularly useful because the process can be highly automated.

---

66. Why Does Let’s Encrypt Verify the Domain?

Suppose someone asks Let’s Encrypt:

```
Give me a certificate for google.com
```

Let’s Encrypt cannot simply issue it.

It first needs evidence that the requester controls the relevant domain.

That domain-control verification is performed through standardized mechanisms.

This is where **ACME** becomes important.

---

67. What is ACME?

ACME stands for:

> **Automatic Certificate Management Environment**

The important thing to remember is:

> ACME is a protocol used for automating certificate issuance and renewal.

Conceptually:

```
Your Server
     |
     | ACME
```

```
     ↓
Let's Encrypt
```

Instead of manually:

- Requesting certificates
- Proving domain ownership
- Downloading files
- Installing certificates
- Renewing certificates

software can automate much of the process.

---

68. Certbot

One of the most commonly used ACME clients is:

> **Certbot**

Certbot can run on your server and communicate with Let’s Encrypt.

Conceptually:

```
Certbot
   |
   | "I need a certificate
   |  for coderarmy.in"
   ↓
Let's Encrypt

Let's Encrypt:
"Prove that you control coderarmy.in."
```

The server then completes a challenge.

---

69. HTTP-01 Challenge

One common ACME challenge is:

HTTP-01

Suppose you request a certificate for:

```
coderarmy.in
```

Let’s Encrypt asks your server to make a specific value available at an HTTP URL similar to:

```
http://coderarmy.in/.well-known/acme-challenge/abc123
```

Let’s Encrypt then makes an HTTP request to that location.

```
Let's Encrypt
       |
       | HTTP Request
       ↓
coderarmy.in
       |
       ↓
Your Server

Expected Challenge Value Found ✓
```

If the expected value is returned:

```
Domain Control Verified ✓
```

Let’s Encrypt can then issue the certificate.

---

70. Why Does HTTP-01 Prove Domain Control?

Suppose:

```
coderarmy.in
```

resolves through DNS to infrastructure that you control.

If you can also make that server respond with a specific challenge value requested by Let’s Encrypt, you demonstrate operational control over the domain.

Conceptually:

```
DNS
 ↓
coderarmy.in
 ↓
Your Infrastructure
 ↓
Correct ACME Challenge Response
 ↓
Domain Control Proven
```

---

71. Basic Certbot Workflow

Suppose:

```
coderarmy.in
      ↓
Your EC2 Public IP
```

and Nginx is already running.

You install:

```
Nginx
Certbot
```

The conceptual workflow becomes:

```
Certbot
   ↓
Contacts Let's Encrypt
   ↓
ACME Challenge
   ↓
Domain Verified
   ↓
Certificate Issued
   ↓
Certificate Downloaded
   ↓
Nginx Configured
```

In common setups, Certbot can even modify Nginx configuration automatically.

---

72. What Certbot Automates

Without automation, certificate management could involve steps such as:

```
Generate CSR
      ↓
Send CSR
      ↓
Verify Domain
      ↓
Download Certificate
      ↓
Copy Certificate Files
      ↓
Configure Nginx
      ↓
Reload Nginx
```

Certbot automates much of this process.

This makes HTTPS deployment much easier and less error-prone.

---

73. Automatic Certificate Renewal

Certificate renewal is one of the most important operational aspects of TLS.

You do not want production infrastructure depending on somebody remembering:

> The certificate expires next Tuesday.

Instead, certificate management should be automated.

Conceptually:

```
Certbot
   ↓
Periodically Check Certificates
   ↓
Certificate Nearing Expiry?
   ↓
Yes
   ↓
Renew Certificate
   ↓
Reload Web Server
```

Good infrastructure removes unnecessary manual operational work.

```
No human remembering certificate dates
                ↓
Automated renewal
```

---

74. Putting Everything Together

Let’s combine the entire networking flow.

Suppose a user enters:

```
https://coderarmy.in/api/products
```

A simplified production flow could look like:

```
Browser
   ↓
DNS Resolution
```

```
   ↓
Public IP
   ↓
Reverse Proxy / API Gateway
   ↓
TLS Termination
   ↓
Authentication / Routing / Rate Limiting
   ↓
Load Balancer
   ↓
Application Instance
```

Before the HTTP request is transmitted:

```
Browser
   ↓
TLS Handshake
   ↓
Server Certificate Verification
   ↓
ECDHE Key Agreement
   ↓
Shared Secret
   ↓
Traffic Keys Derived
   ↓
Secure TLS Channel
```

Then:

```
HTTP Request
     ↓
TLS Encryption
     ↓
Internet
     ↓
TLS Decryption
     ↓
Application
```

And the response follows the reverse path:

```
Application Response
     ↓
TLS Encryption
     ↓
Internet
     ↓
TLS Decryption
     ↓
Browser
```

---

75. Final Mental Model

A useful way to remember the entire topic is:

```
DNS
↓
Where should I connect?

Reverse Proxy / API Gateway
↓
Where should this request go?

Load Balancer
↓
Which application instance should handle it?

TLS
↓
How do we communicate securely?

Certificate
↓
How do I know this server is legitimate?

ECDHE
↓
How do both sides establish shared secrets?

Symmetric Encryption
↓
How do we efficiently protect actual traffic?

Let's Encrypt + ACME + Certbot
```

```
↓
How do we automate certificate issuance and renewal?
```

Quick Revision

Forward Proxy

Represents the **client**.

```
Client → Proxy → Internet
```

Reverse Proxy

Represents the **server side**.

```
Internet → Nginx → Backend
```

API Gateway

Provides a common entry point for application APIs.

```
/api/users    → User Service
/api/products → Product Service
/api/orders   → Order Service
```

HTTP

Defines how web requests and responses are structured.

HTTPS

```
HTTP + TLS
```

Provides:

- Confidentiality
- Integrity
- Authentication

Symmetric Cryptography

```
Same secret key
→ Encrypt
→ Decrypt
```

Fast enough for bulk communication.

Asymmetric Cryptography

```
Public Key
Private Key
```

Useful for authentication and cryptographic key establishment.

Certificate

Binds:

```
Hostname + Public Key + Trust Information
```

Certificate Authority

Issues/signs certificates after performing the required validation.

TLS Handshake

Main goals:

```
Verify Server Identity
        +
Establish Shared Traffic Keys
```

ECDHE

Allows both sides to derive the same shared secret without transmitting that secret directly.

```
Browser: a + Server Public Value
             ↓
        Shared Secret

Server: b + Browser Public Value
             ↓
        Same Shared Secret
```

TLS Traffic

After the handshake:

```
HTTP Request  → Encrypted
HTTP Response → Encrypted
```

TLS Termination

A reverse proxy such as Nginx can handle HTTPS on behalf of backend applications.

Let’s Encrypt

Public Certificate Authority providing automated domain-validated certificates.

ACME

Protocol used to automate certificate issuance and renewal.

Certbot

ACME client commonly used with Let’s Encrypt.

HTTP-01

Domain-control challenge where a specific value must be served from the requested domain.

Certificate Renewal

Should be automated so production HTTPS does not depend on manually remembering expiry dates.